

AI Ops Module 1

Mithilesh — Roll: DA24B048

Question 1: Technical Debt Diagnosis

(a) **Delivery time rounding hurt restaurant feature:** *Entanglement (CACE)*. Changing one feature silently cascaded to an unrelated output because features share hidden statistical dependencies inside the model.

(b) **Marketing team silently reading model output:** *Undeclared Consumers*. External team depends on raw model output without ML team knowledge, creating invisible dependencies that make changes risky.

(c) **14 undocumented shell scripts, no orchestration:** *Configuration & Glue-Code Debt*. Hyperparameters scattered with no single source of truth; the pipeline is unmaintainable glue code.

Mitigation for (c): Replace shell scripts with `dvc.yaml` declaring every stage (preprocess → train → evaluate). Centralise hyperparameters in `params.yaml` and log them to MLflow. Commit both files per run so code, data, and config are always co-located and reproducible via `dvc repro`.

Question 2: MLflow Experiment Comparison

Six MLP experiments were run on MNIST (`load_digits`) varying `batch_size` and `learning_rate` with solver=Adam, max_iter=500, seed=42.

Run	lr	batch	layers	train_acc	val_acc	test_acc
mlp-baseline	0.001	32	(128,64)	0.984	0.962	0.961
mlp-lr-0.01	0.01	32	(128,64)	0.993	0.972	0.958
mlp-lr-0.0001	0.0001	32	(128,64)	0.975	0.948	0.956
mlp-batch-16	0.001	16	(128,64)	0.997	0.976	0.972
mlp-batch-128	0.001	128	(128,64)	0.970	0.922	0.928
mlp-deeper	0.001	32	(256,128,64)	0.982	0.965	0.963

Best run: mlp-batch-16 (lr=0.001, batch=16) achieved val_accuracy=0.976 and test_accuracy=0.972. Smaller batch size increases gradient update frequency per epoch, enabling better weight optimization. **Overfitting:** mlp-lr-0.01 shows train=0.993 vs val=0.972 (gap=0.021), indicating overfitting at high learning rate. mlp-lr-0.0001 underfits with val=0.948, failing to converge in 500 iterations. **Hyperparameter effect:** Batch size has larger impact — changing batch 32→16 improved val_accuracy by 0.014, while changing lr 0.001→0.01 improved it by only 0.010.

MLflow logging code: `mlflow.log_param("learning_rate", lr)` `mlflow.log_param("batch_size", batch_size)`
`mlflow.log_metric("train_accuracy", train_acc)` `mlflow.log_metric("val_accuracy", val_acc)`

Question 4: Capstone — End-to-End Reproducibility

Role: Partner B (Mithilesh, DA24B047). Partner A: Pulkit.

Steps followed as Partner B: (1) `git clone` Pulkit's repo. (2) `git checkout <commit>` to pin the exact commit. (3) `dvc checkout` to restore the Iris dataset. (4) `conda env create -f environment.yml`. (5) Re-ran `python train.py` — MLflow logged all params, metrics, seed, and git-commit tag automatically.

Results: Both metrics matched Partner A exactly within tolerance ± 0.001 .

Metric	Partner A (expected)	Partner B (reproduced)
accuracy	1.0000	1.0000
f1_macro	1.0000	1.0000

Reproducibility note logged in MLflow: Partner B reproduced the result using only `git clone`, `git checkout`, `dvc checkout`, and `conda env create`. Accuracy and f1_macro matched exactly. The protocol — pinned environment.yml, fixed seed=42, MLflow logging, and DVC versioning — was sufficient for 100% metric reproduction. Pulkit's repository and the MLflow run are publicly verifiable. Proof also submitted as a screen recording video.